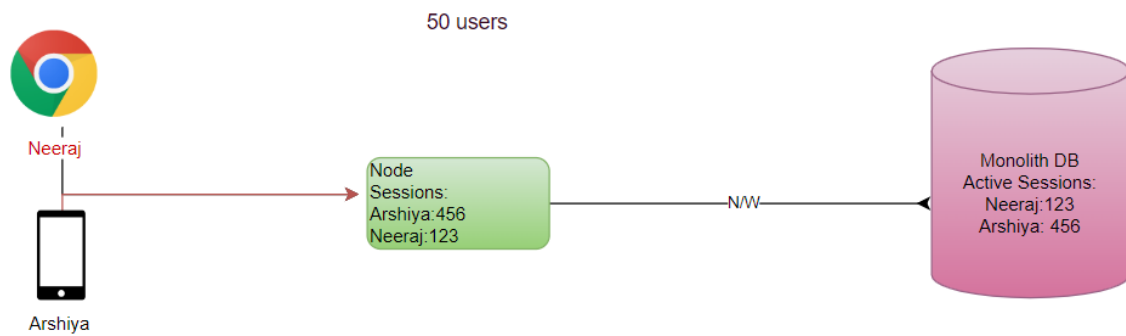


Monolith: all components/modules in one single main component

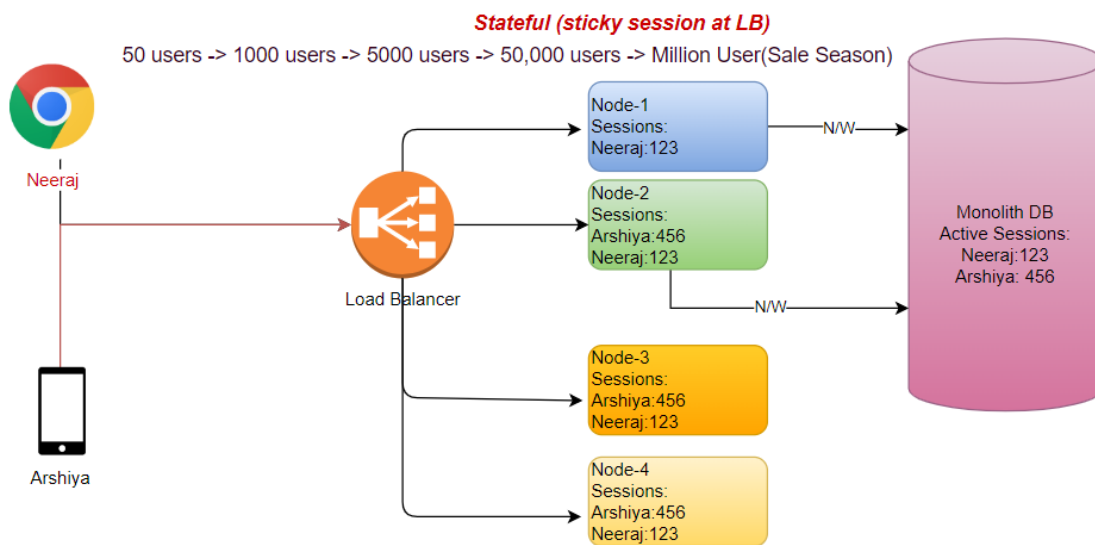
Monolith application (eCommerce Application)

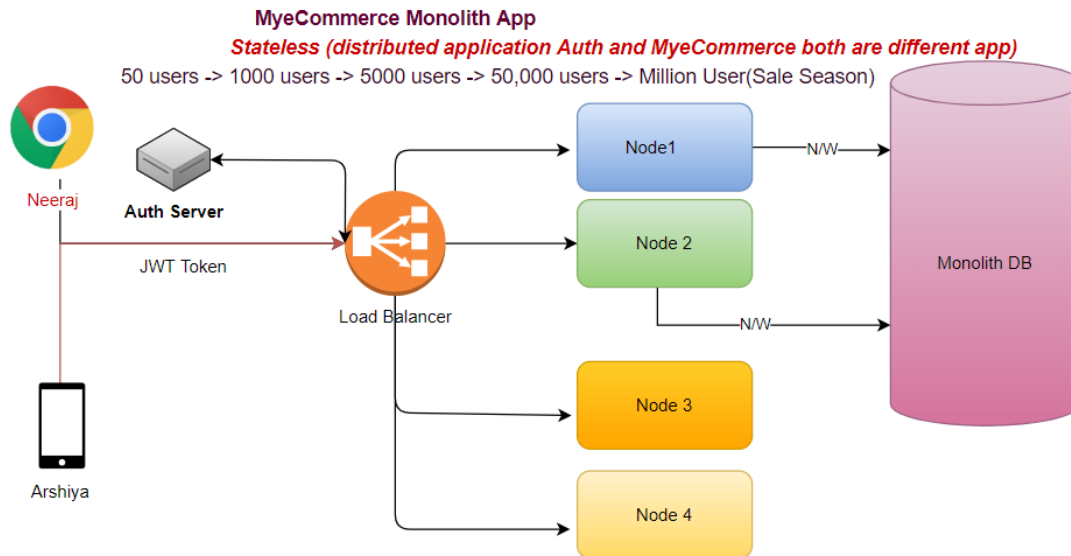


UC: Work load Increasing?

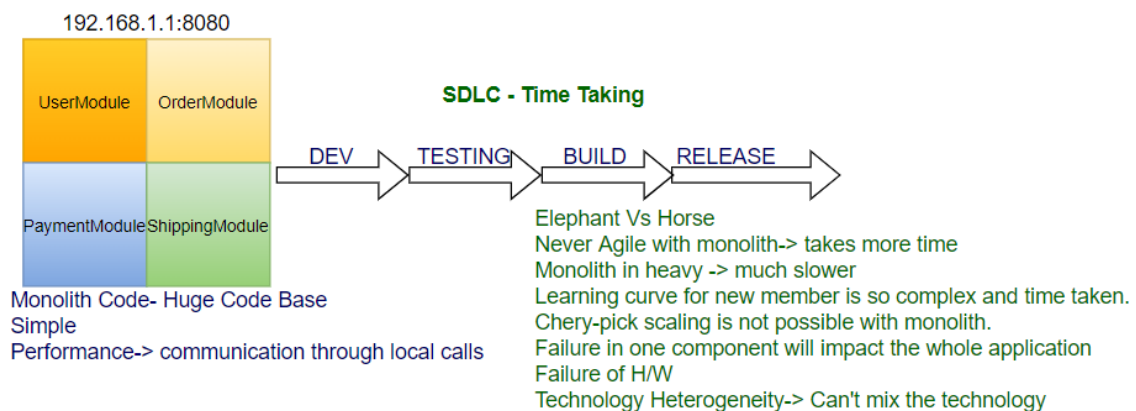
Adding more nodes: Horizontal Scaling

Adding more capacity (H/W): Vertical Scaling (SPF)





- No session management (using tokens user authenticate/authorization)
- Every request has enough info to process the request.



SOA: Service Oriented Architecture

service: task that achieves a functionality

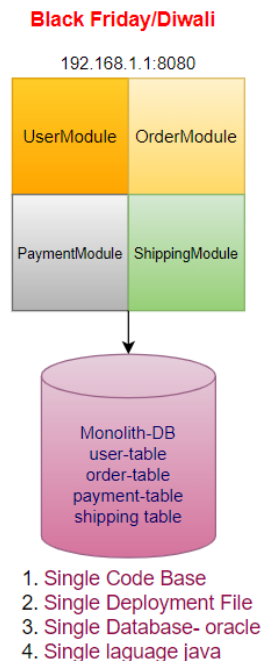
ESB: Enterprise Service Bus (SPF)

- Routing
- Security
- Transformation
- Orchestrating

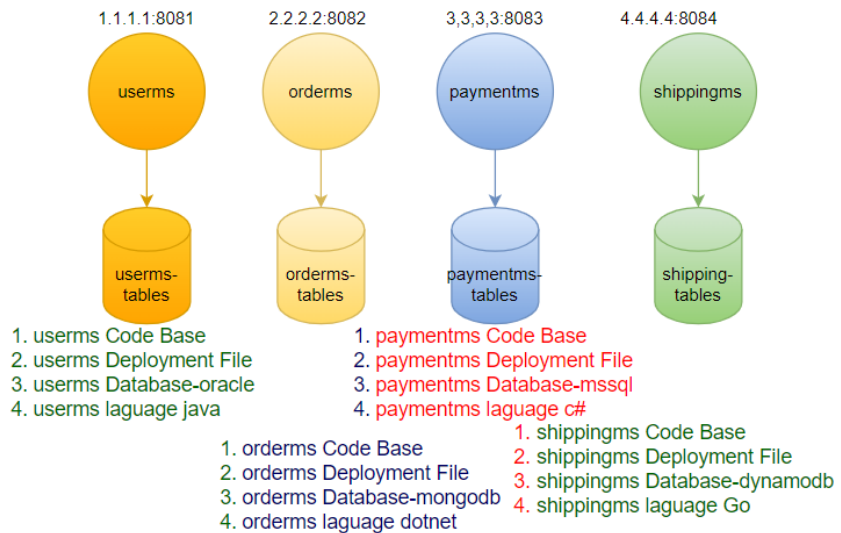
Microservices: Also known as the microservice architecture- is an architectural style that structures an application as a collection of services that are

Difference between Architecture of Monolith/ Microservice.

Monolithic My eCommerce Application



Microservice Architecture



Pros of Microservice:

- Scaling + Cherry pick
- Agility-1 Development fast
- Agility-2 Build fast
- Agility-3 Testing faster
- Agility-4 CI/CD
- Agility-5 Release Faster
- Resiliency
- Distributed Nature (Service Load Distribute)
- Distributed Database Load
- Technology Heterogeneity
- Database Heterogeneity
- Security through Segregation-reduce surface area of attack

Cons of Microservice:

- Latency between ms
- Distributed Data Access (Aggregation/TX Management)
- Complexity in Managing Nodes (servers +DB)
- Cost (Infra+ Developers/DBA)
- Monitoring
- Tracing
- Testing

Building Blocks of Microservices:

- Single Responsibility Principles (High Cohesion)
- Domain Driven Design- Bounded Context
- Communication Synchronous/Asynchronous

Design and Implementation of Microservices?

- The design of microservices basically follows 12 factor principles:
- The implementation of microservice will be developed using spring boot framework:

Microservices 12 factor principles:

1. Single Code base per microservices for all environments (dev, test, prod)-Git
2. All application should bundle their dependencies along with application bundle - Maven
3. Externalize configuration using Configuration Server - Springboot (GitHub)
4. All backing services should be accessible through an addressable URL
5. Strong Isolation between build, release and run ,e.g. DevOps, CI/CD
6. The application should be stateless for high fault tolerance (JWT-session data at client side)
7. A service able to tolerate fault (Replication)
8. Expose service using port bindings
9. Services should be scale out with concurrency
10. Building applications with minimal startup and shutdown times
11. Keeping all environment Identical as Identical as possible.

12. Centralized logging for the microservices (Splunk, ELK, EFK (Fluent Bit))

- a. Logstash- parser, agent
- b. Elasticsearch- Storage
- c. Kibana- Visualization

Spring Boot:

- microservices is not technology specific
- microservice in java can be easily implemented using spring boot

Spring Framework:

Inversion of control:

Factory Design Pattern: take care responsibility of object creation

Application don't take responsibility of object creation; container creates object at run time and give to application.

Objects are created by container with configuration define and give to the application

dependency injection:

Dependent Objects Injected automatically

Spring Core (IOC/DI)- Business Logic

What is springboot:

UC: Develop a webservice/webapp/microservices using spring mvc/spring Rest and JPA.

- Decide which versions of spring mvc, jpa, hibernate to use.
- Which version of jms(activemq)/amqp (RabbitMQ), if application uses messaging systems
- Spring security compatible versions.
- We need to define the configurations (beans/objects) to be created, define the wiring (dependency injection), application context (spring container creates objects-ioc)
- setup hibernate in data layer, creates beans/objects to create hibernate session factory (like connection factory which gives the database connection objects)

- setup logging frame work
- setup caching
- setup monitoring and health check-up
- decide the application server to deploy the application/webservice/microservice
- BUSINESS LOGIC to accomplish all the business requirements.

If you go with spring boot framework efforts reduced by 25%:

Springboot is built on spring only, and spring is built on java.

Springboot is not alternative to spring mvc/spring rest/spring security/spring data

Springboot uses underneath spring modules and make developers tasks easier and faster in setting up framework.

In a summary, spring boot makes easy to create stand alone, production grade spring-based applications that we can just run

@Configuration

@ComponentScan

We are telling to spring container in the @Configuration class to create the objects of the classes that are annotated with @Component with the help of @ComponentScan

@Autowired - for dependency Injection where application contexts contain spring beans through ioc

REST->: Architecture Style of web service which follow http methods

Restful Webservice:

restful api:

api: piece of code that is exposed where the other piece of code can connect

REST-it works on the top of HTTP protocol

- Mainly client-server-based communication
- Client calls the REST API that hosted on the server and server sends the response back
- **Key concepts:**
 - Resources and url's

- Resource: The Object or the entity
- URL: path location of the resource

HTTP Methods:

- GET: http method where client reads data from rest api
- POST: http method where client inserts/creates/sends data to rest api
- PUT: http method where client updates data to rest api
- DELETE: http method where client deletes data through rest api

@Controller- endpoint

@Service- business logic

@Repository - DAO

Springboot:

- With Springboot we can eliminate the dispatcher servlet registrations, @EnableMvc.
- We don't need separate tomcat server, spring boot comes internally with embedded tomcat server
- start.spring.io
- spring initializer creates project template (2.7)
- Spring Web - dependency pom.xml generated by Initializer
- spring boot comes with starter
- @SpringBootApplication=@Configuration+@ComponentScan+@EnableAutoConfiguration

Springboot main features:

- Auto Configuration
- Starter Packs with version management
- tomcat server (embedded)
- provides quicker development, comes with more than 40 different starter modules, these starter modules provide integration libraries to many frameworks such as a database connection, webservices, logging, testing libraries (Mockito,), Logging, Template rendering. e.g., for web and mvc related pack is spring-boot-starter-web, for data and jpa, the starter related pack is spring-boot-starter-data.
- When you build a project, when we need to build a spring boot with mvc, security, jdbc then we just add

- spring boot parent module-2.7.0
- spring-boot-starter-web
- spring-boot-starter-jdbc
- spring-boot-starter-security

@EnableAutoConfiguration:

- Will create the bean automatically if it sees the libraries in the class path, for example if we include spring-boot-starter-web in the pom.xml, automatically
- servlet dispatcher, handler mapping, default view resolver is automatically created.
- Will look for libraries in the class path and creates beans automatically
- This will auto configure the required components if their dependencies are on the class path, example: if we have spring-boot-starter-web as a dependency in pom.xml (that means it is in classpath) then web mvc is automatically configured (dispatcher servlet registration, handler mappings, default view resolver, json to java and java to json conversions are automatically configured).

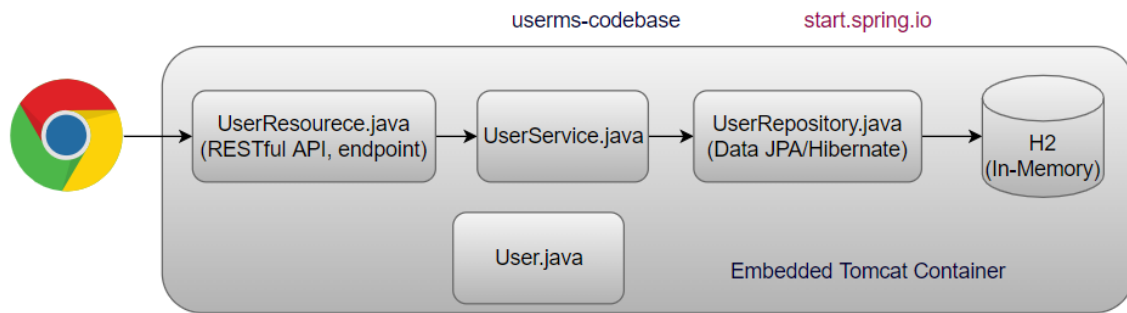
Microservices:

- Spring boot is the library or framework where we can implement microservices easily
- Speed in development
- Easily integrated with devops and agile model
- Containerized model- each microservice runs in its own server
- Springboot with help of autoconfiguration makes the config setup quickly
- Spring boot comes with tomcat server and it can be easily containerized.

The thumb rule:

- If we need to develop 3 microservices, that means practically we need to develop 3 spring boot projects
- Each springboot project:
 - controller (defining the rest api end points)
 - service (business logic)
 - repositories or DAO layer (data access operations)

Flow:



Concepts:

- Service Registry and Discovery- Netflix Eureka
- Client-Side Load Balancing- Netflix Ribbon/Spring Cloud LoadBalancer
- Resiliency Implementation Using Circuit Breaker- Hystrix/Resilience4j
- Monitoring Using Actuator and Micrometre - Prometheus/Grafana
- API tracing using spring cloud sleuth and Zipkin
- Centralized Config Using Spring Cloud Config
- Centralized logging using ELK
- Security Using OAuth and JWT
- API Gateway- Zuul/Spring Cloud Gateway

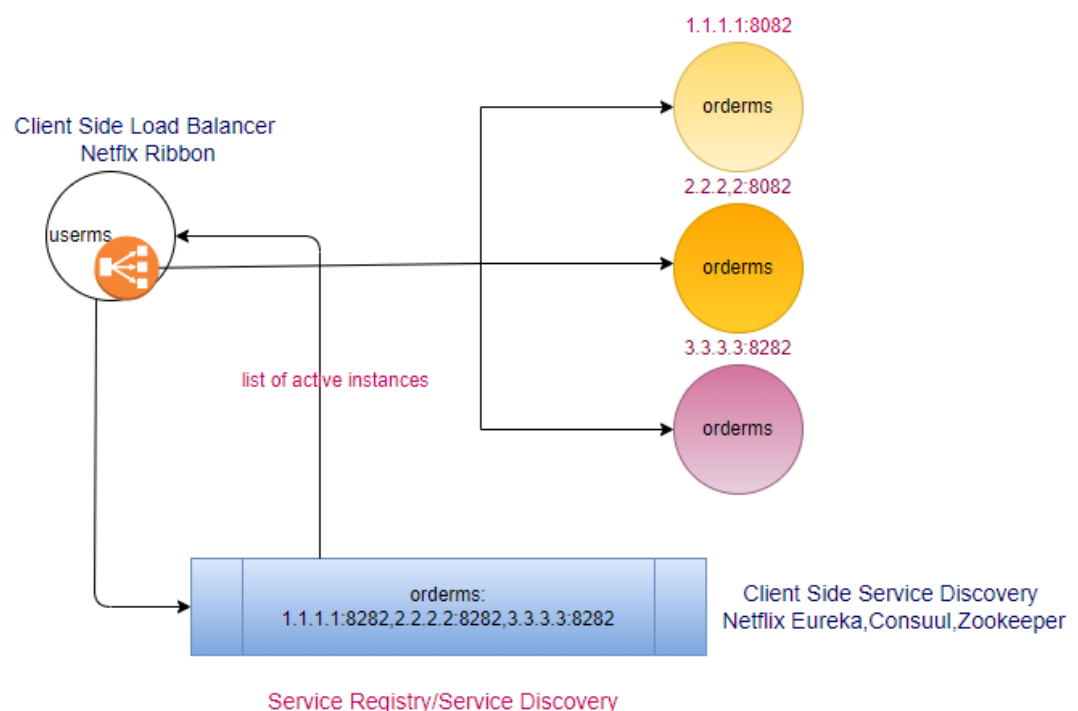
Env:

- eclipse latest
- java 17
- mysql
- maven 3
- arc- advance rest client
- tomcat 10
- Git Client
- activemq
- kafka
- aws cli
- WinSCP

UC: users wants to call orderms, but ip address of ms is not static its dynamic in nature, how to find the ip address of orderms?

Solution: Service Discovery/Service Registry, just like telephone directory, it keeps track of each ms.

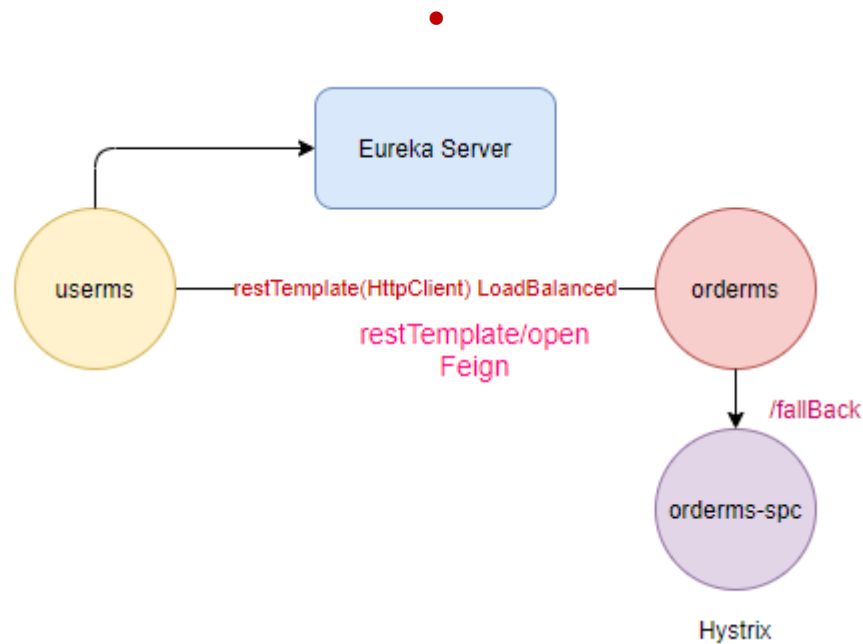
- Register each ms in **eureka registry**, each instance of ms sends their heart beat to the eureka server.
- server doesn't look for, instance itself sends the heartbeat to the eureka registry
- if instance down, stops to send the heartbeat, eureka registry remove that instance details from the registry.
- It is not for load balancing, just only for discovering services dynamically.



- If there are more than one instance of orderms available then which instance will be called?

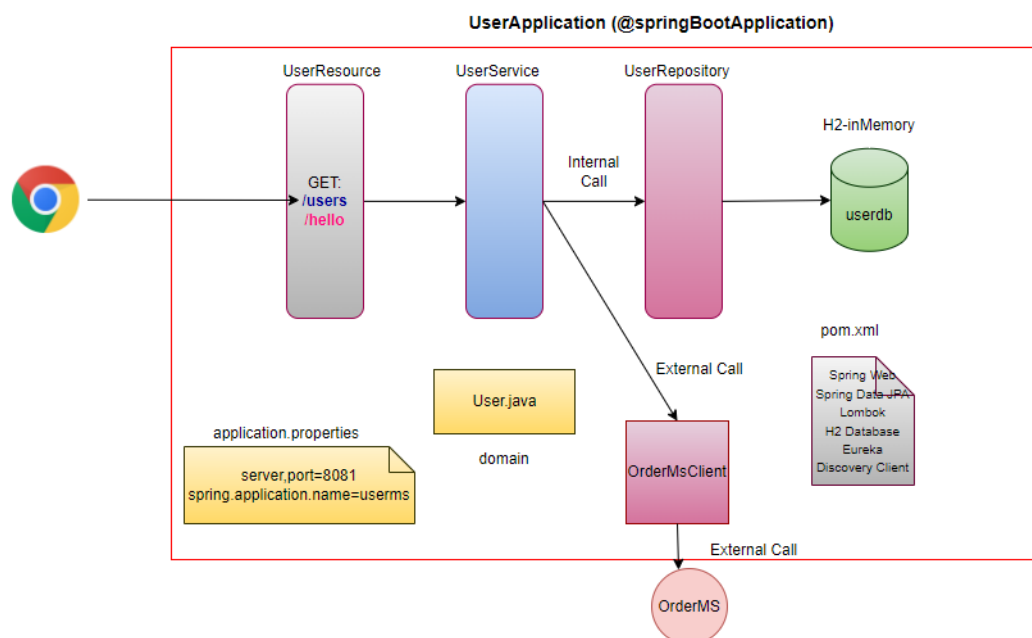
Client-Side Load Balancer: Inside the source ms (Ribbon)

- First time ribbon takes values from the registry after that **it keeps in cache (TTL)** provided by eureka and calls the orderms, getting reference from cache directly.



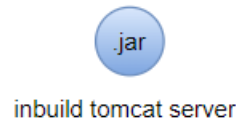
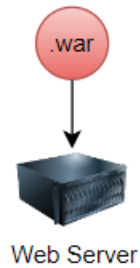
@SpringBootApplication

- @EnableAutoConfiguration- scan all classpath .jar on your behalf
- @ComponentScan



.jar Vs .war

Make .jar Not .war # making .jar is cloud native approach for building microservices, .war requires external container. .jar are self-contained



Project
☒ Maven Project
☐ Gradle Project

Language
☒ Java ☐ Kotlin ☐ Groovy

Spring Boot
☐ 3.0.0 (SNAPSHOT) ☐ 3.0.0 (M4) ☐ 2.7.4 (SNAPSHOT) ☒ 2.7.3
☐ 2.6.12 (SNAPSHOT) ☐ 2.6.11

Project Metadata

Group com.freddie

Artifact usersms

Name usersms

Description Demo project for Spring Boot

Package name com.freddie.usersms

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

H2 Database

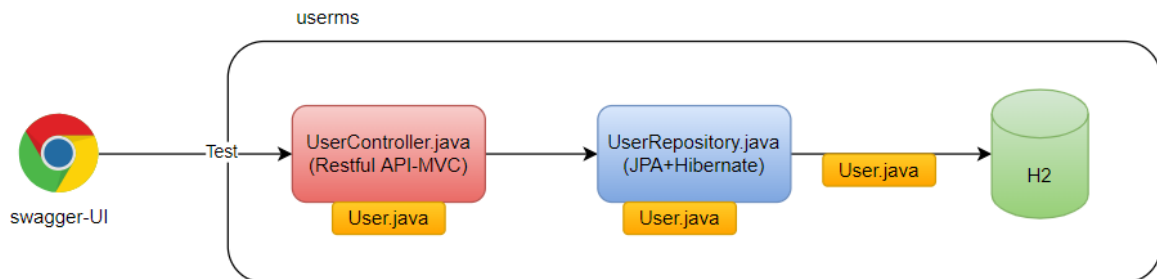
SQL

Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

Spring Data JPA

SQL

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.



```
C:\Users\neera\Downloads\usersms>dir
Volume in drive C has no label.
Volume Serial Number is B00E-A64B

Directory of C:\Users\neera\Downloads\usersms

14-09-2022  17:22    <DIR>          .
14-09-2022  17:22    <DIR>          ..
14-09-2022  17:22                395 .gitignore
14-09-2022  17:22    <DIR>          .mvn
14-09-2022  17:22           1,043 HELP.md
14-09-2022  17:22          10,284 mvnw
14-09-2022  17:22           6,734 mvnw.cmd
14-09-2022  17:22           1,490 pom.xml
14-09-2022  17:22    <DIR>          src
               5 File(s)          19,946 bytes
               4 Dir(s) 385,098,092,544 bytes free

C:\Users\neera\Downloads\usersms>mvn -version
Apache Maven 3.8.6 (84538c9988a25aec085021c365c560670ad80f63)
Maven home: C:\Software\apache-maven-3.8.6
Java version: 17.0.4, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-17.0.4
Default locale: en_IN, platform encoding: Cp1252
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"

C:\Users\neera\Downloads\usersms>mvnw clean install -Dmaven.test.skip=true
```

mvnw clean package -Dmaven.test.skip=true

Import maven project in Eclipse

@SpringBootConfiguration

@EnableAutoConfiguration

@ComponentScan

In Spring framework

@ComponentScan ("com. freedie. myus"), Initialize bean on our behalf

- @Component
- @Service
- @Controller
- @RestController
- @Entity
- @Configuration
- @Repository # Is optional from spring 5 onwards

@Bean # Method level

```
public MyJavaClass initializeMyJavaClass(){  
    MyJavaClass myJavaClass=new MyJavaClass();  
    myJavaClass.setMyProperty("abc");  
    return myJavaClass;  
}
```

@EnableAutoConfiguration

Will scan the lib directory , automatically establish default configuration if not available in properties file.

localhost:port/h2-console

localhost:port/swagger-ui/

Project

☒ Maven Project

☐ Gradle Project

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 3.0.0 (SNAPSHOT)

☐ 3.0.0 (M4)

☐ 2.7.4 (SNAPSHOT)

☒ 2.7.3

☐ 2.6.12 (SNAPSHOT)

☐ 2.6.11

Project Metadata

Group

com.freddie

Artifact

eureka-server

Name

eureka-server

Description

Demo project for Spring Boot

Package name

com.freddie.eureka-server

Dependencies

ADD DEPENDENCIES... CTRL + B

Eureka Server

SPRING CLOUD DISCOVERY

spring-cloud-netflix Eureka Server.

Sleuth

OBSERVABILITY

Distributed tracing via logs with Spring Cloud Sleuth.

Project

☒ Maven Project

☐ Gradle Project

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 3.0.0 (SNAPSHOT)

☐ 3.0.0 (M4)

☐ 2.7.4 (SNAPSHOT)

☒ 2.7.3

☐ 2.6.12 (SNAPSHOT)

☐ 2.6.11

Project Metadata

Group

com.freddie

Artifact

orders

Name

orders

Description

Demo project for Spring Boot

Package name

com.freddie.orders

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Eureka Discovery Client

SPRING CLOUD DISCOVERY

A REST based service for locating services for the purpose of load balancing and failover of middle-tier servers.

Project

☒ Maven Project

☐ Gradle Project

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 2.7.0 (SNAPSHOT)

☐ 2.6.3 (SNAPSHOT)

☐ 2.6.2

☐ 2.5.9 (SNAPSHOT)

☒ 2.5.8

Project Metadata

Group

com.edu

Artifact

apigateway

Name

apigateway

Description

Demo project for Spring Boot

Dependencies

ADD DEPENDENCIES... CTRL + B

Sleuth

OBSERVABILITY

Distributed tracing via logs with Spring Cloud Sleuth.

Eureka Discovery Client

SPRING CLOUD DISCOVERY

A REST based service for locating services for the purpose of load balancing and failover of middle-tier servers.

Gateway

SPRING CLOUD ROUTING

Provides a simple, yet effective way to route to APIs and provide cross cutting concerns to them such as security, monitoring/metrics, and resiliency.

Spring Boot Actuator

OPS

Supports built in (or custom) endpoints that let you monitor and manage your application - such as application health, metrics,